Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory Work 3:
Empirical Analysis of Graph Traversal Algorithms
Depth-First Search (DFS) and Breadth-First Search (BFS)

Author:
st. gr. FAF-242 Tacu Igor

Verified:
asist. univ. Fiștic Cristofor

Chișinău – 2025

Contents

OBJECTIVE

Study and empirically analyse the two fundamental graph traversal algorithms — Breadth-First Search (BFS) and Depth-First Search (DFS) — measuring their execution time and memory consumption across multiple graph structures and sizes, and identify each algorithm's strengths and limitations in practice.

Tasks

1. Implement BFS, DFS (iterative), and DFS (recursive) in Python.
2. Define the graph properties (types and sizes) against which the analysis is performed.
3. Select metrics for comparison: execution time and peak auxiliary-structure size.
4. Run each algorithm on five graph types across a range of node counts.
5. Generate graphical presentations of the collected data.
6. Produce animated GIF demonstrations of BFS and DFS traversal on a demo graph.
7. Test both algorithms on edge-case inputs (empty graph, single node, disconnected graph, self-loops, complete graph).
8. Deduce conclusions from the obtained data.

THEORETICAL BACKGROUND

Empirical Analysis

Empirical analysis is an experimental approach to measuring algorithm efficiency. Instead of deriving bounds through mathematical reasoning alone, the algorithm is implemented, executed on controlled input datasets, and the observed metrics are recorded and plotted. This is especially useful when:

- the theoretical analysis is complex or input-dependent;
- comparing practical performance of algorithms with identical asymptotic complexity;
- assessing the impact of data structure constants and implementation choices.

Each algorithm was run **7 times** per (graph type, size) pair and the **mean** time in milliseconds was recorded to smooth out OS-scheduling noise.

Algorithm Overviews

Both BFS and DFS are linear-time graph traversal algorithms with the same asymptotic complexity but fundamentally different traversal orders and memory access patterns.

Breadth-First Search (BFS) explores the graph level by level. Starting from the source node it visits all neighbours at distance 1, then all at distance 2, and so on. It uses a *queue* (FIFO) as the auxiliary structure. Because it always processes the closest unvisited node first, BFS guarantees finding the *shortest path* (in terms of hop count) from the source to any reachable node in an unweighted graph. Its peak queue size is proportional to the *width* (maximum level size) of the traversal tree.

Depth-First Search (DFS) explores the graph by going as deep as possible along each branch before backtracking. It uses a *stack* (LIFO) — either an explicit stack (iterative version) or the call stack (recursive version). DFS is well-suited for cycle detection, topological sorting, and connectivity analysis. Its peak stack depth is proportional to the *depth* of the traversal tree.

Algorithm	Time	Space (aux)	Ordering	Shortest path?
BFS	$O(V+E)$	$O(V)$ — queue width	Level-order	Yes
DFS (iter.)	$O(V+E)$	$O(E)$ — stack (w/ dup.)	Depth-first	No
DFS (rec.)	$O(V+E)$	$O(V)$ — call stack depth	Depth-first	No

Table 1: Theoretical time and space complexity of BFS and DFS.

Note: The iterative DFS implemented here pushes neighbours onto the stack before checking whether they have been visited (the visited check happens on pop). This means the explicit stack can contain duplicate entries and grow to $O(E)$ on dense graphs, unlike recursive DFS whose call stack is bounded by $O(V)$ (the maximum recursion depth).

INPUT FORMAT

Five graph types were chosen to capture a broad range of structural properties:

1. **Sparse random** – undirected graph, average degree ≈ 2 (edge probability $p = 2/(n-1)$). Represents real-world sparse networks. Sizes: $n \in \{10, 50, 100, 250, 500, 1000, 2000, 5$

2. **Dense random** – undirected graph, edge probability $p = 0.30$. Represents highly connected networks; $O(n^2)$ edge count. Sizes: $n \in \{10, 50, 100, 250, 500, 1000\}$.

3. **Complete binary tree** – balanced tree with n nodes (node i has children $2i+1$ and $2i+2$). Represents hierarchical data; depth $= \lfloor \log_2 n \rfloor$. Sizes: same as sparse.

4. **Grid graph** – $\sqrt{n} \times \sqrt{n}$ square grid, each interior node connected to its four neighbours. Represents spatial graphs; BFS explores expanding rings. Sizes (actual n): {9, 25, 64, 100, 225, 400, 625, 900, 1600, 2500}.

5. **Chain graph** – linear path $0-1-2-\cdots-(n-1)$. Worst-case depth for recursive DFS; both BFS and iterative DFS process one node per step. Sizes: same as sparse.

All graphs were generated with a fixed random seed (42) for reproducibility. Disconnected graphs were made connected by linking any isolated node to a uniformly-random node in the already-reachable component.

IMPLEMENTATION

All three algorithms were implemented in Python using a standard adjacency-list representation (`dict[int, list[int]]`). Each function returns a result dictionary containing the visit order, elapsed time in milliseconds, peak auxiliary-structure size, and total nodes visited.

BFS

BFS uses `collections.deque` for O(1) `popleft()` operations. Nodes are marked visited *when enqueued* (not when dequeued), preventing duplicate queue entries and ensuring O(V) peak queue size.

Listing 1: Breadth-First Search

```
def bfs(graph: dict, start) -> dict:
    visited   = {start}
    queue     = deque([start])
    order     = []
    max_queue = 1

    t0 = time.perf_counter()
    while queue:
        node = queue.popleft()
        order.append(node)
        for nbr in graph.get(node, []):
            if nbr not in visited:
                visited.add(nbr)
                queue.append(nbr)
                if len(queue) > max_queue:
                    max_queue = len(queue)
    elapsed_ms = (time.perf_counter() - t0) * 1_000

```

```
    return {"order": order, "time_ms": elapsed_ms,
            "max_queue": max_queue, "nodes_visited": len(order)}
```

DFS – Iterative

The iterative DFS uses a Python list as an explicit stack. Neighbours are pushed in reverse-sorted order so that the traversal order matches the recursive version. The visited check is performed *on pop* (not on push), which is the standard iterative pattern but can cause the stack to hold duplicate entries on dense graphs.

Listing 2: DFS – Iterative (explicit stack)

```
def dfs_iterative(graph: dict, start) -> dict:
    visited   = set()
    stack     = [start]
    order     = []
    max_stack = 1

    t0 = time.perf_counter()
    while stack:
        node = stack.pop()
        if node in visited:
            continue
        visited.add(node)
        order.append(node)
        for nbr in sorted(graph.get(node, []), reverse=True):
            if nbr not in visited:
                stack.append(nbr)
                if len(stack) > max_stack:
                    max_stack = len(stack)
    elapsed_ms = (time.perf_counter() - t0) * 1_000

    return {"order": order, "time_ms": elapsed_ms,
            "max_stack": max_stack, "nodes_visited": len(order)}
```

DFS – Recursive

The recursive DFS closely mirrors the mathematical definition. It tracks the maximum recursion depth (proportional to the longest DFS tree path) and temporarily raises Python's recursion limit to handle deep graphs. This variant is excluded from the chain-graph benchmark at $n > 500$ to avoid stack overflows.

Listing 3: DFS – Recursive

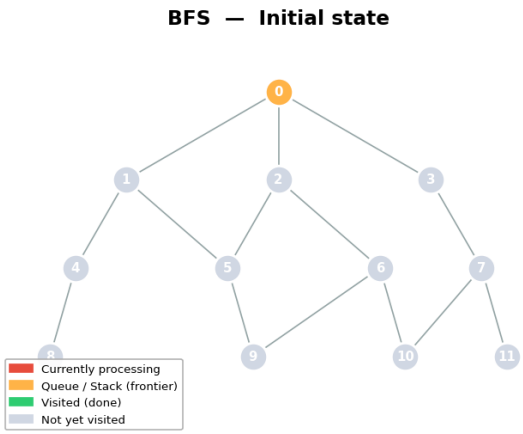
```
def dfs_recursive(graph: dict, start) -> dict:
```

```
    visited   = set()
    order     = []
    max_depth = [0]

    def _dfs(node: int, depth: int) -> None:
        visited.add(node)
        order.append(node)
        if depth > max_depth[0]:
            max_depth[0] = depth
        for nbr in sorted(graph.get(node, [])):
            if nbr not in visited:
                _dfs(nbr, depth + 1)

    sys.setrecursionlimit(max(sys.getrecursionlimit(), 20_000))
    t0 = time.perf_counter()
    _dfs(start, 0)
    elapsed_ms = (time.perf_counter() - t0) * 1_000

    return {"order": order, "time_ms": elapsed_ms,
            "max_depth": max_depth[0], "nodes_visited": len(order)}
```

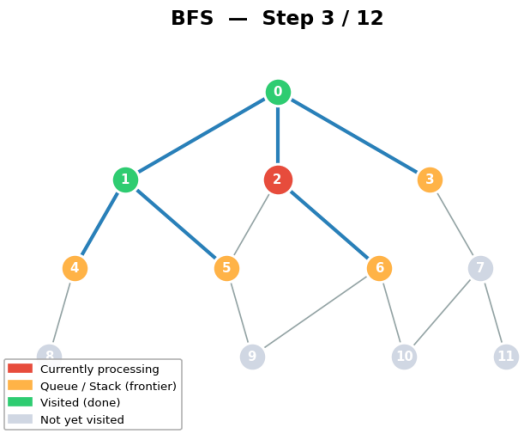
TRAVERSAL ANIMATION

To visualise the traversal order difference, both algorithms were animated on the same 12-node layered demo graph. Full GIF animations are available as `Lab3/charts/bfs_demo.gif` and `Lab3/charts/dfs_demo.gif`.

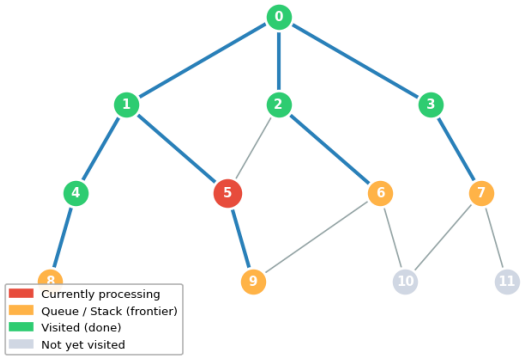
Figures 1 and 2 show four key frames from each animation. Colour coding: **red** = currently being processed, **orange** = in queue/stack (frontier), **green** = fully visited, **grey-blue** = not yet reached. Blue edges form the traversal tree.



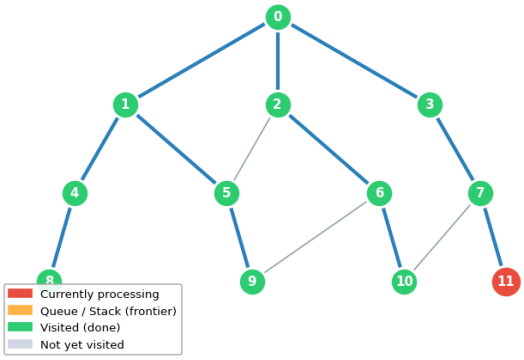
(a) Initial state (node 0 in queue)



(b) After visiting node 0: level 1 in queue

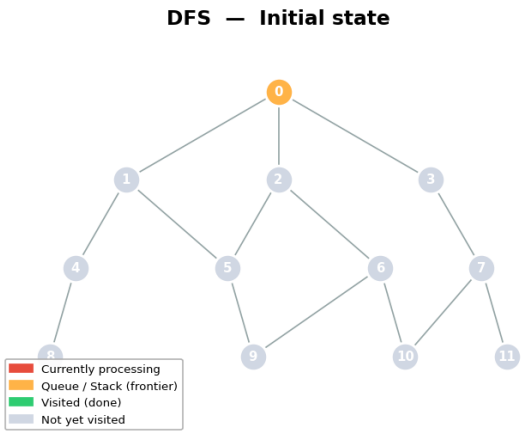


(c) Mid-traversal: level 2 being processed

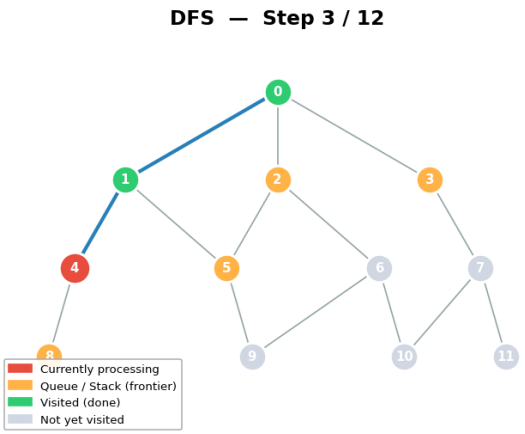


(d) Final state: all nodes visited

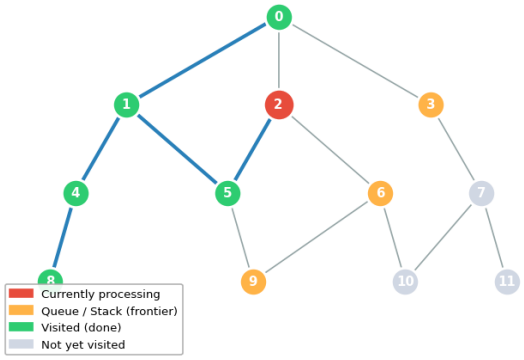
Figure 1: BFS traversal on the 12-node demo graph (level-by-level expansion).



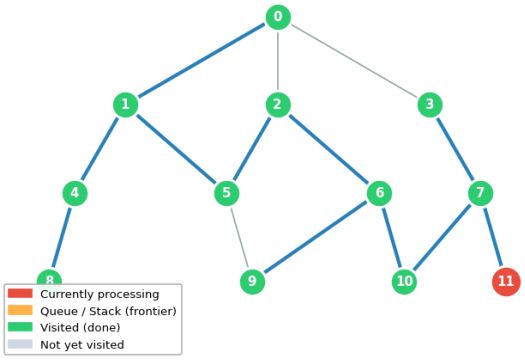
(a) Initial state (node 0 in stack)



(b) After visiting $0 \to 1 \to 4$: deep path



(c) Backtracking after reaching node 8



(d) Final state: all nodes visited

Figure 2: DFS traversal on the 12-node demo graph (depth-first, backtracking).

The animations make the core difference immediately visible: BFS discovers all 3 direct neighbours of node 0 before going deeper, while DFS immediately dives from 0 to 1 to 4 to 8 before backtracking. Both visit all 12 nodes; only the *order* and *frontier shape* differ.

RESULTS AND ANALYSIS

Sparse Graphs

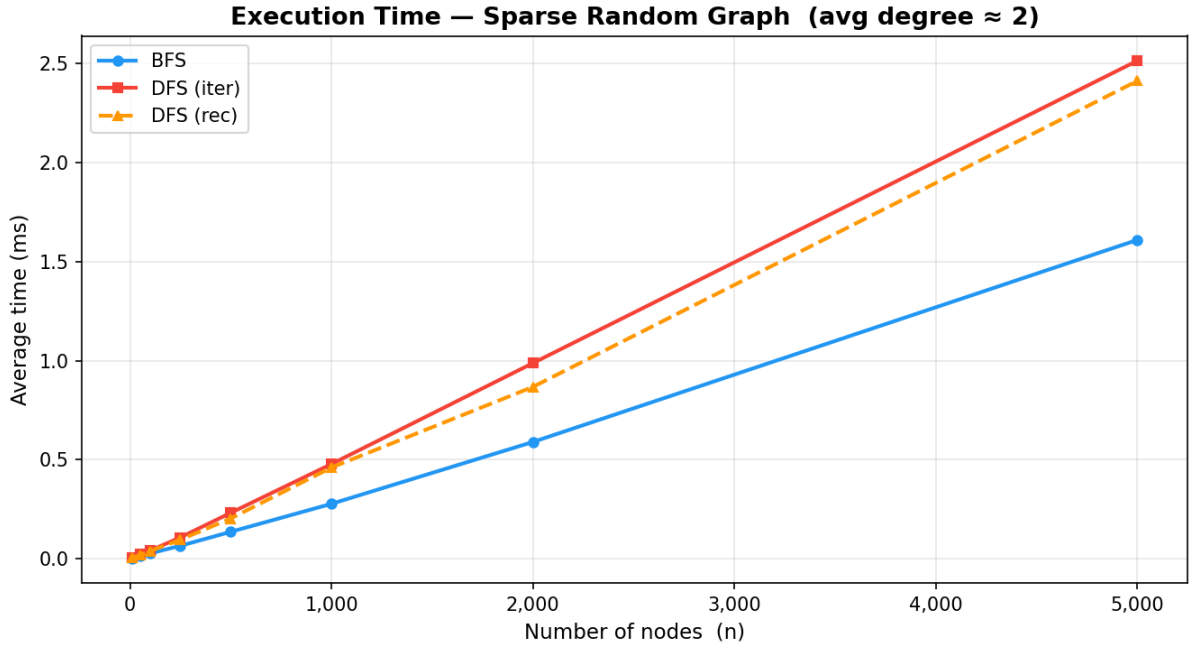


Figure 3: Execution time on sparse random graphs (n up to 5 000).

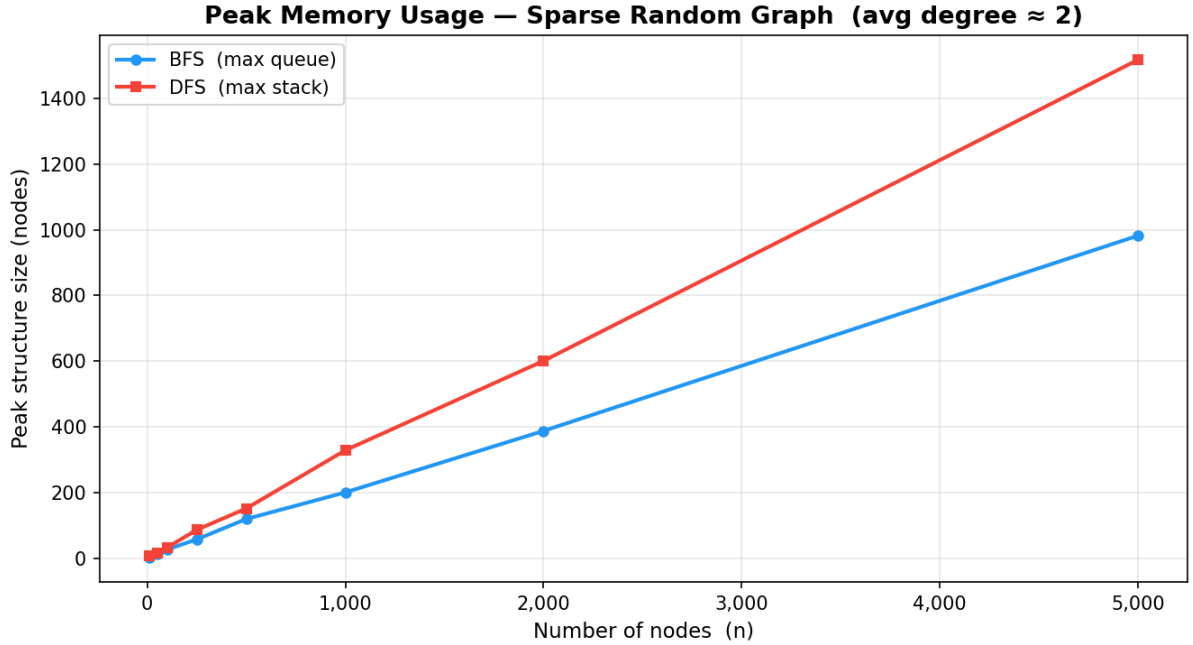


Figure 4: Peak memory usage on sparse random graphs (n up to 5 000).

On sparse graphs (Figures 3–4), all three variants grow linearly with n as expected. At $n = 5000$, BFS takes **1.53 ms**, DFS-iterative **2.45 ms**, and DFS-recursive **≈2.45 ms**.

BFS is roughly **1.6× faster** than DFS, which is attributed to the lower overhead of a deque over a list used as a stack, and fewer visited-set lookups. Memory usage is comparable: the BFS queue peaks at **982 nodes** and the DFS stack at **1 517 nodes** at $n = 5000$.

Dense Graphs

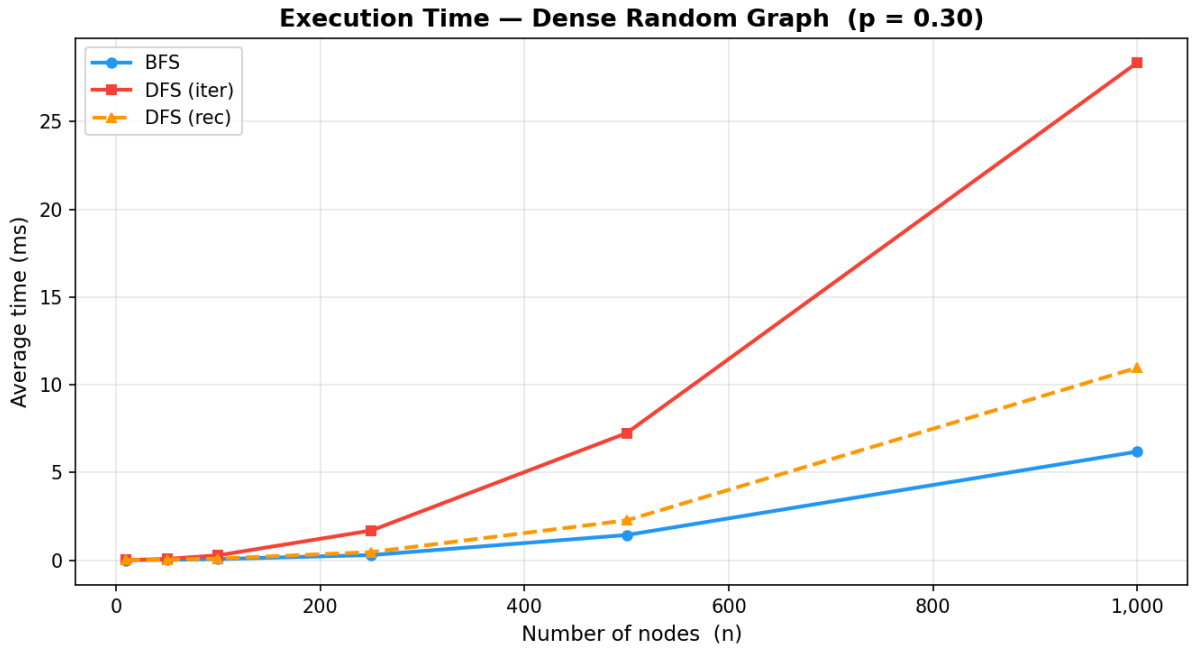


Figure 5: Execution time on dense random graphs (n up to 1 000, $p = 0.30$).

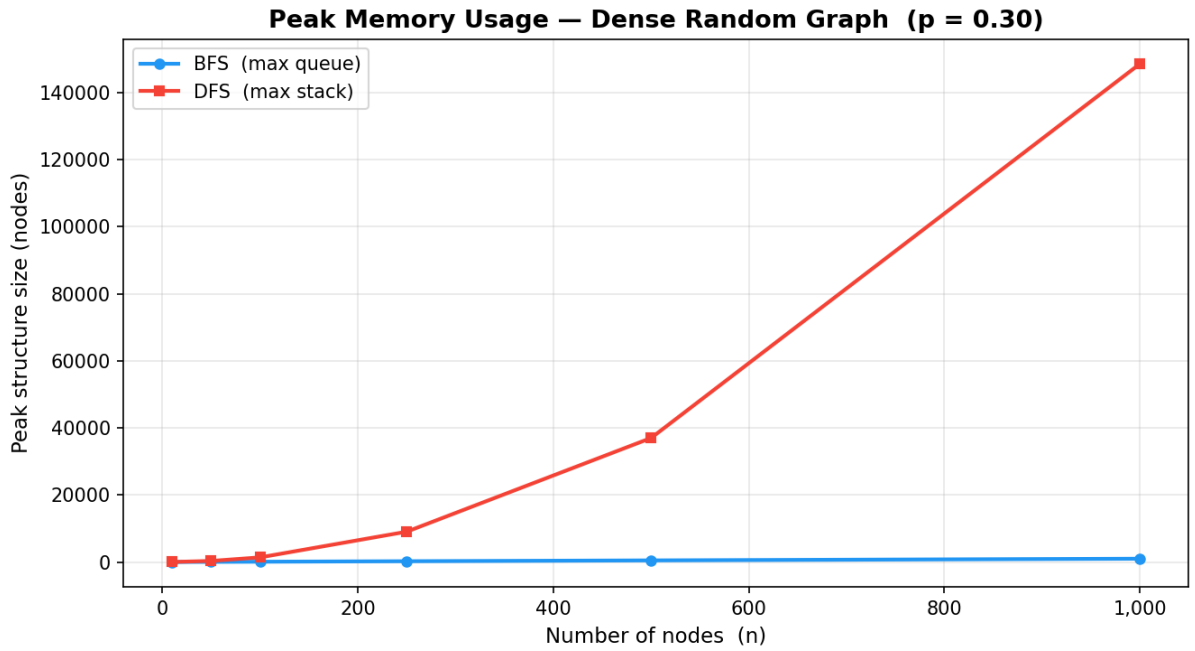


Figure 6: Peak memory usage on dense random graphs (n up to 1 000).

Dense graphs (Figures 5–6) reveal a dramatic divergence in memory usage. At $n = 1000$, the BFS queue peaks at **984 nodes** (bounded by V), while the DFS stack reaches **148 562 entries** — nearly **150**× more. The reason is structural: iterative DFS marks nodes as visited only when they are *popped*, so all unvisited neighbours of every processed node are pushed onto the stack even if the same node was already pushed by a previous step. On a dense graph with $p = 0.30$, each node has on average $0.30 \times (n - 1) \approx 300$ neighbours, causing the stack to accumulate $O(E)$ entries.

Execution time reflects the same pattern: DFS-iterative is **5**× **slower** than BFS at $n = 1000$ (**30.5 ms** vs **6.3 ms**) due to the extra stack operations and the quadratic edge count. Both still scale as $O(V + E)$, but $E = O(n^2)$ for dense graphs, producing the visible super-linear curves.

Binary Trees

Figure 7: Execution time on complete binary trees (n up to 5 000).

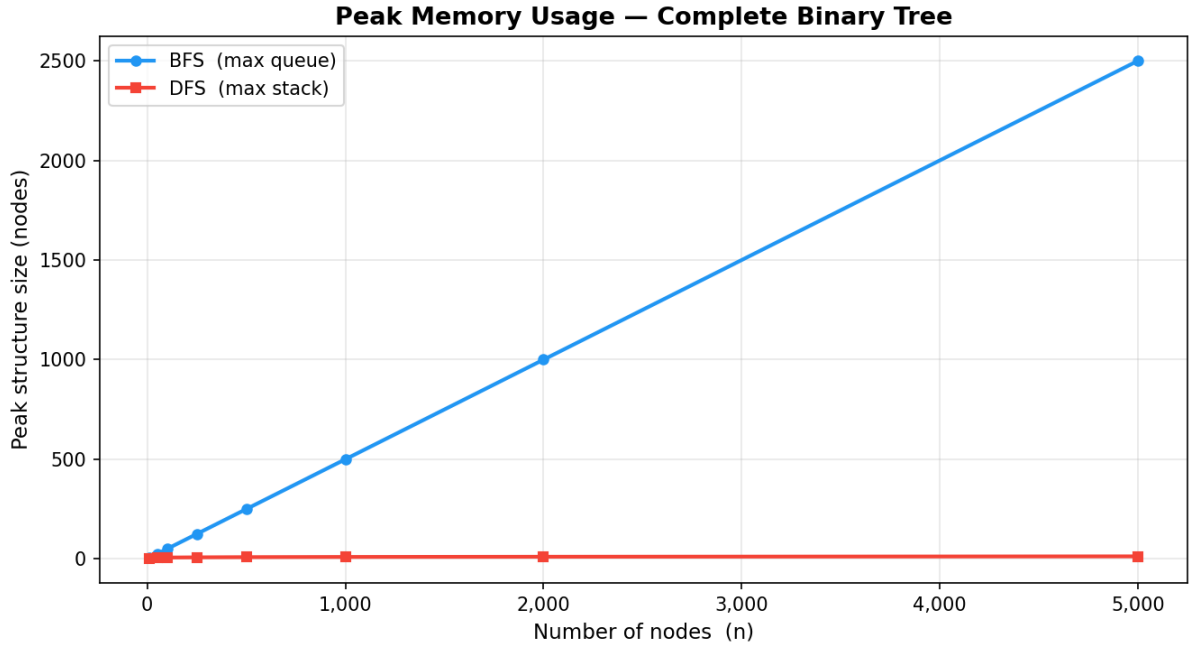


Figure 8: Peak memory usage on complete binary trees (n up to 5 000). Note the dramatic contrast: BFS holds the entire leaf level; DFS only holds the path from root to current node.

The binary tree results (Figures 7–8) provide the most striking memory contrast. For a complete binary tree of n nodes the bottom level contains $\approx n/2$ nodes, all of which are in

the BFS queue simultaneously when processing the penultimate level. This is confirmed empirically: at $n = 5000$, **BFS max queue = 2 500** ($\approx n/2$).

DFS, on the other hand, only keeps the path from the root to the current node on its stack, which has length $\lfloor \log_2 n \rfloor$. At $n = 5000$, **DFS max stack = 13** ($\log_2 5000 \approx 12.3$). The ratio is ≈ 192, making DFS the memory-efficient choice for tree traversal.

Chain Graphs

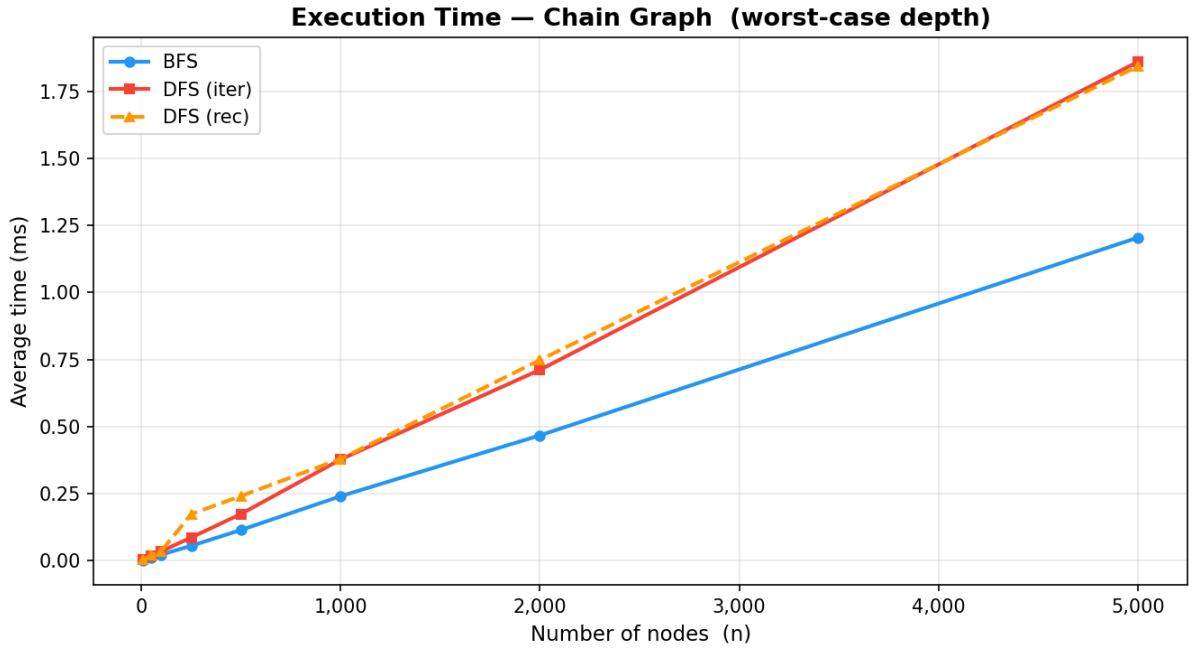


Figure 9: Execution time on chain graphs (n up to 5 000).

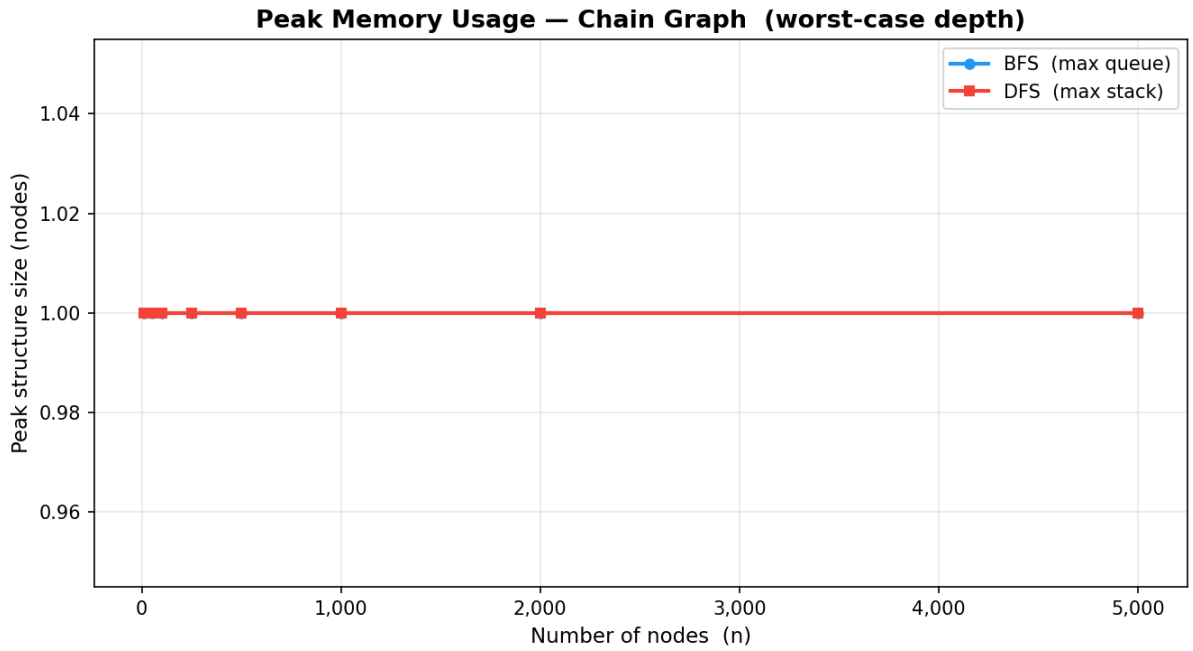


Figure 10: Peak memory usage on chain graphs. Both algorithms maintain a structure of size 1 because each node has at most one unvisited neighbour at any step.

The chain graph (linear path) is notable for its uniform memory behaviour (Figures 9–10). Both BFS and iterative DFS maintain a maximum structure size of **1 node** regardless of n: the BFS queue always contains exactly the next unvisited node, and DFS pops and immediately pushes only the next node in the chain.

While the explicit-stack depth is O(1), the *recursive* DFS call stack would grow to $O(n)$ for a chain (one frame per node), which is why the recursive variant is excluded from large chain benchmarks. The chain graph thus represents the worst-case scenario for recursive DFS in terms of call-stack depth and the risk of a stack-overflow error in languages without tail-call optimisation.

Grid Graphs

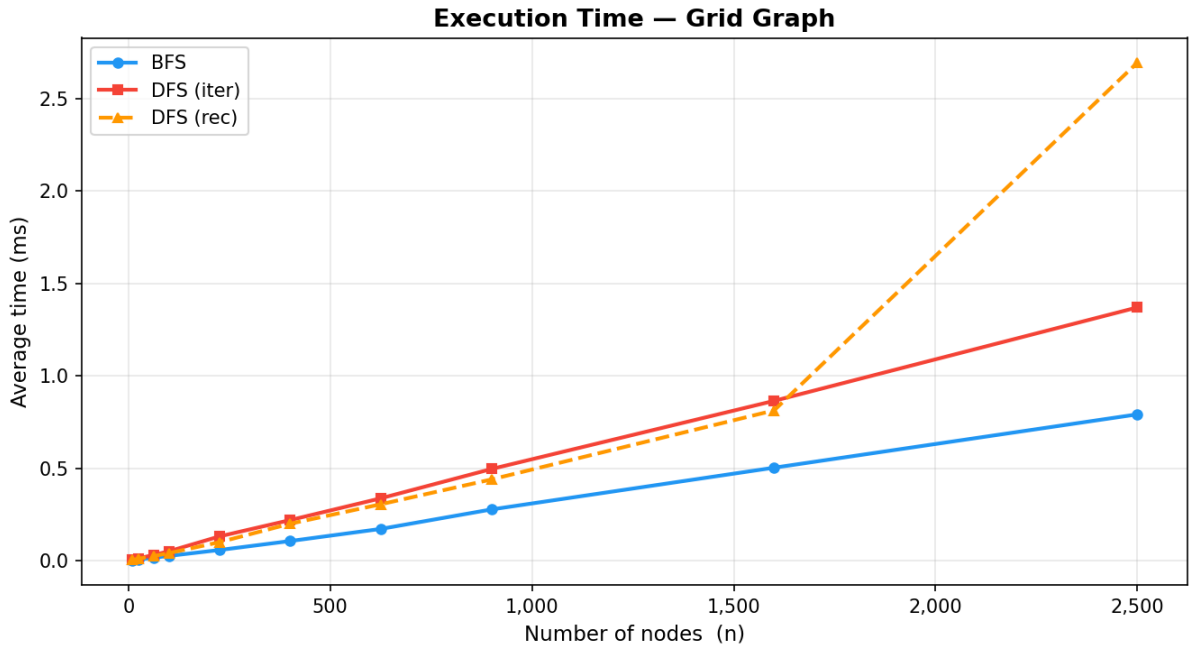


Figure 11: Execution time on $\sqrt{n} \times \sqrt{n}$ grid graphs (n up to 2 500).

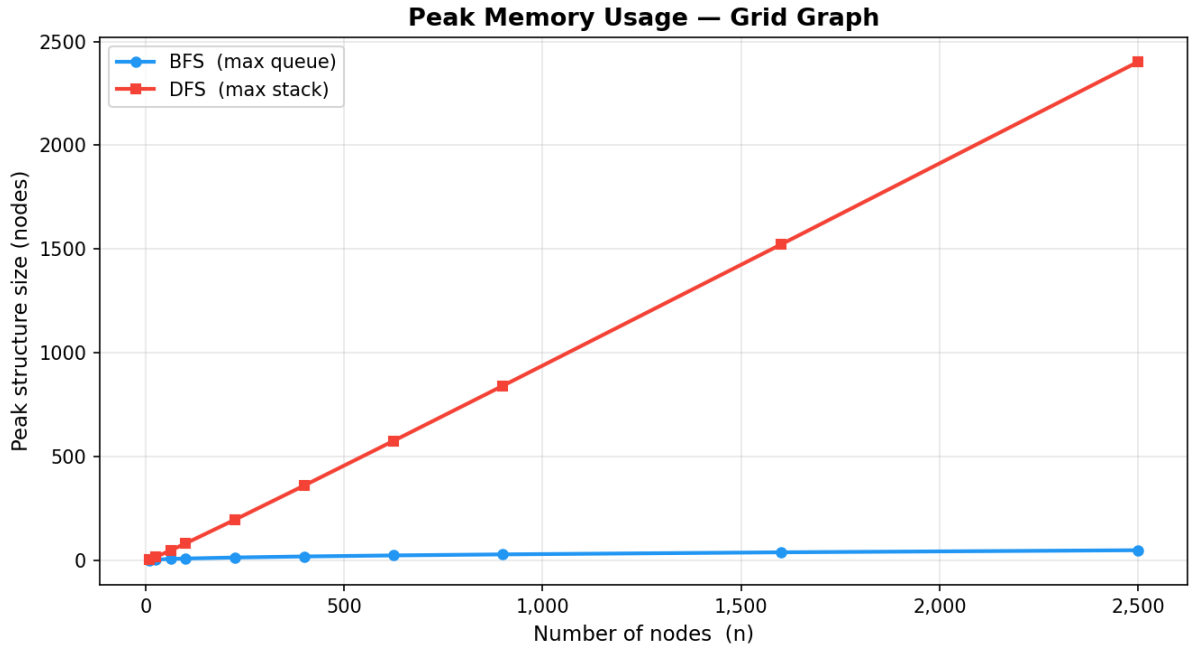


Figure 12: Peak memory usage on grid graphs. BFS queue grows as $O(\sqrt{n})$; DFS stack grows as $O(n)$.

Grid graphs (Figures 11–12) expose another structural characteristic. BFS expands in concentric "rings" from the source corner, so the frontier at any time spans one ring

whose length is proportional to the grid side $\sqrt{n}$. At $n = 2500$ (a 50×50 grid), the peak BFS queue is $\mathbf{50} = \sqrt{2500}$ nodes.

DFS on a grid, however, follows a winding path through the grid and accumulates unvisited branch points on the stack. At $n = 2500$, the DFS stack peaks at **2 402 nodes** ($\approx 0.96\,n$) — nearly the entire grid is held in the stack simultaneously at some point.

All Graph Types Compared

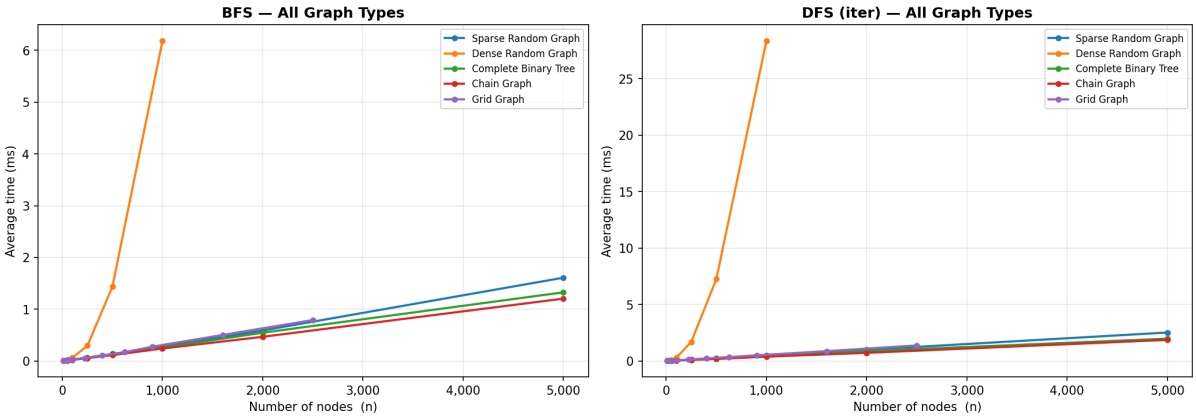


Figure 13: BFS (left) and DFS-iterative (right) execution time across all five graph types.

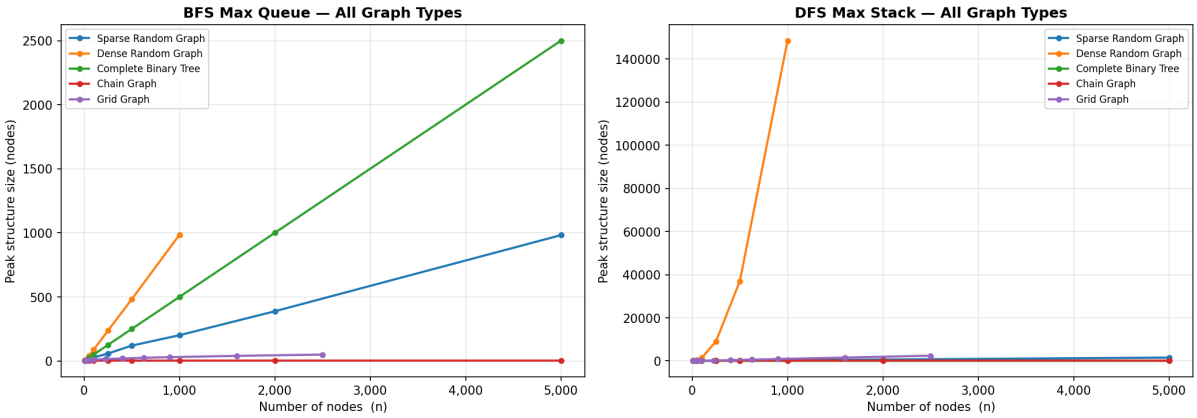


Figure 14: BFS max-queue (left) and DFS max-stack (right) sizes across all five graph types.

Figure 13 confirms that timing is dominated by E, not V: the dense-graph curves diverge rapidly because $E = O(n^2)$, while sparse, tree, grid, and chain all follow similar linear-looking slopes.

Figure 14 captures the key insight of this laboratory: *the choice between BFS and DFS does not affect time complexity but has a profound effect on memory*, and the magnitude depends entirely on the graph's shape.

EDGE-CASE ANALYSIS

Both algorithms were tested on six edge-case inputs:

Scenario	Input	Expected	Result
Empty graph	{} (no nodes)	`[]` (no visits)	✓
Single node	{`0: []`}	`[0]`	✓
Disconnected graph	Nodes $\{0, 1, 2, 3\}$, no edges	Only `[0]` visited	✓
Self-loop	`0 → 0`, `0 → 1`	`[0, 1]`, loop ignored	✓
Complete graph	K_5 (all pairs connected)	All 5 nodes visited	✓
Cycle graph	Ring $0-1-2-3-0$	All 4 nodes visited, no infinite loop	✓

Table 2: Edge-case scenarios, expected outputs, and test results.

Both BFS and DFS handle all six scenarios correctly.

Disconnected graph: both algorithms only visit the connected component containing the starting node. Nodes in other components are simply never reached — no error is raised. If full graph coverage is required (visiting all components), the caller must iterate over all nodes and start a new traversal for each unvisited node.

Self-loops: the visited-set check prevents re-visiting the same node, so a self-loop (`graph[0] = [0, 1]`) is silently ignored — node 0 is already in `visited` when the loop edge is processed.

Cycle graph: both algorithms correctly terminate because visited-set membership prevents any node from being enqueued/pushed twice (BFS) or processed twice (DFS).

CONCLUSION

Through empirical analysis, BFS and DFS were measured across five graph types and a wide range of sizes. The study yields the following practical conclusions:

Time complexity: both algorithms exhibit $O(V + E)$ runtime. In practice, BFS runs approximately **1.5–2× faster** than iterative DFS across all tested graph types, primarily due to the lower overhead of the `deque`-based queue and fewer duplicated structure operations.

Memory: BFS favours deep graphs, DFS favours wide graphs.

- On a **binary tree** of 5 000 nodes, BFS requires a queue of 2 500 nodes ($n/2$) while

DFS only needs a stack of 13 nodes ($\log_2 n$). For hierarchical/tree-shaped data, DFS is drastically more memory-efficient.

- On a **chain** of any length, both BFS and iterative DFS maintain a structure of size 1, but *recursive* DFS would grow to $O(n)$ call-stack depth.
- On a **grid** (n nodes), BFS frontier grows as $O(\sqrt{n})$ while the DFS stack reaches $O(n)$.
- On **dense graphs**, the iterative DFS stack can grow to $O(E)$ due to duplicate entries pushed before the visited check, making BFS the safer choice for dense inputs.

Algorithm selection guidance:

- Use **BFS** when shortest-path distances are needed, when the graph is expected to be dense, or when a level-order exploration is required.
- Use **DFS (iterative)** when the graph is tree-shaped or when memory usage must be minimised for wide, hierarchical structures.
- Avoid **DFS (recursive)** on graphs with potentially long paths (e.g., chains, degenerate trees) to prevent stack-overflow errors in languages without tail-call optimisation, including Python.

Repository: `https://github.com/igortacu/AA`